CANDY
DIALOGUE ENGINE

LLM

1.1.0

LLM Integration

Candy DE has built-in features that make it usable for dialogue generated by an LLM (Large Language Model).

To avoid any misunderstanding, Candy DE does not provide a way to connect your game to an LLM: instead you must implement that on your own. Thankfully, it's quite simple and, from our own testing, an LLM can explain how to do it in just 30 minutes.

Candy DE isn't picky: any API or LLM model will do, and it can be a local API or an online one. You could even build an API into your game if you felt like it. All that matters, as far as Candy DE is concerned, is that your game is able to pass prompts to an LLM and receive outputs.

Notice: Candy Arts now provides an addon for connecting your games to an LLM API of your choosing - Candy LLM. It also comes with example code for the llm_query() function described in this document.

LLM Support

What Candy DE does provide is the following:

In the settings (**Candy_Settings.gd**) you can set the **llm_mode** variable to 'true'. This will make Candy DE attempt to use LLM-generated text for every spoken line.

In **Candy_Database.gd**, in the actors_dictionary, you can also modify the value of an actor's "LLM" key to determine whether LLM-generated text is used for that actor. -1 means that LLM-generation is never used (even if **llm_mode** is enabled), 1 means that it is always used, and 0 complies with the default settings (**llm_mode**).

Finally, you can also indicate for each spoken line whether LLM-generation should be used. This is done in the "LLM Override" data field, in Candy Dialogue Creator. Note that the line's settings override both the default settings and the actor's.

If any of these settings indicate that a line should have its text generated by an LLM, Candy DE will call the **llm_query()** function.

llm_query()

The **llm_query()** function, located in **Candy_Functions.gd**, is called when Candy DE processes a spoken line which is meant to be generated by an LLM.

When your game is connected to an LLM, **llm_query()** should send a prompt to the LLM, receive an output in return, and return that output to Candy DE. Very simple.

Crafting Prompts

You'll probably want to tailor the prompts a little bit, to get good results from an LLM.

When Candy_DE calls **llm_query()**, it passes relevant information as arguments: the speaker's name, the speaker's reference in the **actors_dictionary** (in case you want to collect more data for the prompt), the actor's disposition (may provide relevant context), the line's text (which you can use as a prompt), and the name of the chosen variant (for example, in case you want to use an 'LLM prompt' variant of that variant).

Typically, LLM prompts consist of general instructions. For example:

"You are Alice, an 18 year old student (...). John is a friend of Alice. Alice thanks John for helping her study. Write what Alice says to John in 2 or 3 sentences."

Depending on the LLM model you use, you may have to add tags, do a bit of formatting, phrase things differently...

For added context, which can help LLMs write more accurate responses, you can look for earlier dialogue lines in the saved dialogue history, if you haven't disabled it: **candy_ui.dialogue_history** (**Candy_Variables.gd**).

Basically, write some code that can create a suitable prompt from various pieces of dialogue data. For best results, consider using slightly different logic for different LLM models, and recommend a few models that work well to your players.

This code can be written inside **llm_query()** or in other functions that llm_query() will call.

After a prompt has been created, send it to the LLM.

Reviewing Outputs

The LLM will return a response (commonly called 'output') automatically to whatever function you wrote that sent the prompt to it.

If your prompts are simple, basically asking for a couple sentences of dialogue, and if the model being used isn't too bad, the responses will be decent enough to use most of the time.

However, you can implement a bit of review logic if you'd like. We can't provide examples or go into details here, but you might want to write a function that checks the formatting, length, strange characters (might indicate an error) or checks for blacklisted words.

If such a function flags an output as containing problems, you could code the function to resend a prompt to the LLM. If you repeatedly encounter errors in outputs, your prompts may be improperly designed, or the LLM/API may be incorrectly configured.

If you feel like delving into algorithms, you could write code that can edit LLM outputs, such as fixing formatting, removing incorrect symbols, or replacing blacklisted words with acceptable synonyms.

Displaying LLM Responses

In any case, once you have a satisfying output from the LLM, **llm_query()** should return it to Candy DE.

At the end of **llm_query()**, just add: '**return llm_output**' (or whatever variable the output is stored in).

Candy DE will then display that output, exactly as returned, in place of the spoken text for the line.

That's all there is to it. As you can see, on the Candy DE side of things: it's all very simple. The real work is getting your game to connect to an LLM API, and writing code to craft at least a basic an prompt based on the dialogue context.

It's not as complicated as it may seem and, as mentioned, an LLM can be very helpful: from personal experience, even without any knowledge of LLMs and connecting to APIs, you can have a basic, working setup in 30 minutes to one hour.